

Predicting NVIDIA Stock Price Direction

Asma Begum
Begum.36@wright.edu

Abstract—This paper discusses how to use machine learning to predict whether NVIDIA stock prices will rise or fall each day. The goal is to understand stock market behavior, which is a challenging task. The problem is simplified by framing it as a yes or no question: will the price increase or decrease tomorrow? Stock data is analyzed and specific techniques are applied to enhance its usefulness, and the focus is on identifying whether the price is trending up or down in both the short and long term while also examining volatility. A Random Forest classifier is used to identify patterns in the data, as it performs well with complex and noisy information. Its performance is evaluated to understand how effective it is and whether it can support decision-making. The results show that predicting the stock market is difficult, but the method is able to uncover useful patterns that can help in making informed decisions about NVIDIA stock.

Keywords—Machine Learning, Stock Prediction, Random Forest, Feature Engineering, Financial Data, Classification Model, Time Series Analysis, Model Evaluation

GitHub Link: https://github.com/asmabegum7233/ML_Project-.git

I. INTRODUCTION

Accurately predicting stock market movements is still a challenge in finance. This is because market data can change a lot, is noisy, and does not follow a linear pattern. Many factors affect stock prices, such as investor sentiment, economic conditions, and unexpected global events. This makes it hard to forecast reliably. Knowing if stock prices will go up or down is very useful for traders, investors, and financial institutions to make good decisions and manage risks.

Recently, machine learning has become popular for predicting financial outcomes because it can handle complex data patterns. Traditional methods like linear regression do not work well with stock prices because they assume linear relationships. Newer methods such as advanced machine learning models and deep learning techniques have been explored to improve predictions. However, many studies either focus only on predicting prices or use complex models without clearly understanding the underlying features. This creates a gap where simpler models with well-designed features can provide effective solutions.

This work focuses on predicting the movement of NVIDIA stock. A machine learning framework is used for this purpose. Instead of using raw price values, historical data is transformed into useful features that represent momentum and market volatility. A Random Forest classifier is selected because it handles noisy data well and can model complex interactions without requiring extensive tuning.

The main contributions of this paper are:

- The study introduces a machine learning method centered on features to forecast stock price movements, leveraging momentum and volatility indicators.
- A Random Forest algorithm is employed to model complex patterns in financial data while minimizing the risk of overfitting.
- Model performance is assessed using accuracy, F1 score, AUC-ROC, backtesting procedure.
- An examination of feature importance helps identify which indicators most affect predictions, enhancing the model's transparency and interpretability.

This study shows that despite the uncertainty of financial markets, carefully designed features and robust machine learning models can provide valuable insights for short-term stock prediction. NVIDIA stock is used as the primary case study.

II. RELATED WORK

A. Supervised Learning Approaches

Supervised learning is commonly applied to stock market forecasting, with models trained on labeled historical data to predict future price changes. Popular methods include Logistic Regression, Support Vector Machines (SVM), Decision Trees, and ensemble techniques like Random Forest and Gradient Boosting. Research [1], [2] suggests that ensemble models typically deliver better performance than simpler linear models, as they can effectively model non-linear patterns and complex feature interactions.

However, many existing approaches depend primarily on raw price values or basic technical indicators, which may fail to fully capture the complexities of market behavior. Moreover, certain models tend to overfit when dealing with the high noise levels typical of financial datasets. This study addresses these limitations by using engineered features such as price ratios and volatility indicators instead of raw prices. The goal is to build a model that performs consistently across different market conditions. Random Forest is chosen because it not only achieves good accuracy but also handles noisy and irregular financial data better than a single decision tree.

B. Self-Supervised and Unsupervised Methods

Unsupervised learning methods can find patterns in stock market data without using any labels. Clustering pulls similar data points together, while PCA strips away extra noise by shrinking variables down. Autoencoders do something similar but rebuild the input after squeezing it tight. When there's no clear target to chase, these tools help spot trends that might otherwise stay buried. Since there is no target label, the model finds structure in the data on its own.

Still, these techniques rarely fit well when it comes to forecasting direction, since labels like "up" or "down" aren't naturally generated. Though certain research efforts tried merging unlabeled feature learning with labeled classification models, but the results did not improve much because the patterns found were hard to interpret. For this study, using supervised learning with clear up and down labels works better, as it keeps results grounded and easier to evaluate.

C. Foundation Models and Transfer Learning

Recent advances in machine learning have introduced large pretrained models that can reuse knowledge across different tasks, commonly seen in speech and image recognition. When predicting market trends, earlier trained systems plus complex network designs like LSTM and those built on Transformers are now used to track patterns over time in sequential data [3].

Even though such models perform well, they usually need massive data, heavy computing power, along with precise adjustments to settings. Their complicated nature tends to make outcomes harder to understand - especially critical when used in finance. Moving away from intense deep learning methods, this work turns toward simpler, clearer models that demand less computation. By using carefully designed features with a Random Forest classifier, this project achieves reasonable performance without requiring large pretrained models or heavy computing resources.

III. METHODOLOGY

A. Problem Formulation

This work treats stock forecasting as a two-class setup under supervision. Suppose x lies in $\mathbb{R}$ raised to d . Here, x represents the input feature vector where d is the number of engineered features derived

from historical stock data. Price momentum along with market volatility make up these characteristics. The symbol y belongs to the set $\{0,1\}$, pointing to the outcome being predicted. When y equals 1, the stock value rises the following trading session. A zero means either a drop happens or no shift occurs at all.

The goal is to learn a function f_θ that maps the input features x to the predicted label y . The model is trained on historical data so that it can predict future stock price direction based on past patterns. The parameter θ is adjusted during training to make predictions as accurate as possible.

Training happens through reduction of a specific objective function

$$L(\theta) = \sum_{i=1}^n \ln(f_\theta(x_i), y_i) + \lambda R(\theta)$$

Among the components, $\ell(\cdot)$ stands for a loss function quantifying differences between predictions and true labels; meanwhile, $R(\theta)$ adds constraints to limit excessive complexity. Balancing these parts, λ adjusts how much emphasis goes toward clean generalization instead of perfect training alignment.

In a Random Forest, this loss function is not directly visible but works through Gini impurity at each split. Instead of limiting tree growth manually, minimum sample rules are applied at each node to control complexity.

Because it pulls broad trends from past information, the method keeps the system from fixating on random fluctuations - a key advantage when forecasting movements in finance.

B. Model Architecture

The proposed model is based on a Random Forest classifier, an ensemble learning technique that combines multiple decision trees to improve predictive performance and robustness. Unlike deep neural networks that use layered neurons, Random Forest builds many separate decision trees where each tree learns patterns independently and the final prediction is made by combining all their votes.

As shown in Figure 1, the model consists of several key components described below.

- **Input Layer:**
The input to the model is a feature vector $x \in \mathbb{R}^d$
In this analysis, d equals 3. Ratio10 captures brief price movements, while Ratio50 reflects extended patterns; both combine with measures of volatility. These features were chosen because they capture both short- and long-term price trends along with market volatility.
- **Group of Tree Decisions:**
One hundred decision trees form the foundation of this approach. From the data, each tree draws its own bootstrapped version selected randomly with repeats allowed - to stand apart from the others. At each split, only a random subset of features is considered, which keeps the tree different from each other. As a result, the overall prediction gains strength through varied perspectives.
- **Tree Structure and Splitting Criteria:**
Splitting happens step by step, where every node divides data using thresholds that reduce disorder. Because class separation matters, Gini impurity guides each choice about where to cut. As branches form, the structure captures complex patterns without relying on straight lines.
- **Regularization Mechanisms:**
Putting limits on how trees grow helps avoid fitting too closely to random variations. For instance, requiring at least fifty observations before splitting a node can restrict excessive branching. Such rules guide the algorithm toward broader patterns instead of minor irregularities.

- **Aggregation Layer with Voting Mechanism:**
Each tree produces a prediction of either 0 or 1. The final output is determined by majority voting across all 100 trees.

What sets this structure apart from neural nets lies in its lack of defined hidden layers or activation mechanisms. Rather than relying on such components, it builds complexity by stacking decision thresholds inside every tree.

Putting many simpler models together helps handle messy real-world finance numbers without getting too complicated. Because of this setup, the system stays accurate while adapting to unpredictable shifts in pricing trends. Starting simple keeps results clearer, even when data acts unexpectedly.

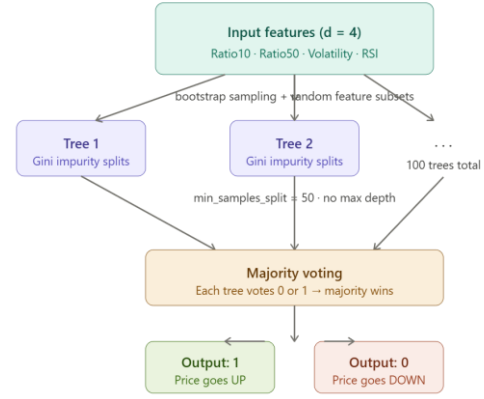


Fig. 1. Architecture overview of the proposed model.

C. Training Procedure

The training process follows a tree-based ensemble approach rather than traditional gradient-based optimization. Since the Random Forest model does not rely on backpropagation, concepts such as optimizers, learning rate schedules, and epochs are not applicable. Instead, the model learns by constructing multiple decision trees using randomized sampling and feature selection.

The training is performed using a bagging strategy, where each tree is trained on a bootstrap sample of the dataset. This introduces diversity among trees and helps improve generalization. A chronological split is applied to maintain time-series integrity, with 80% of the data used for training and the remaining 20% for testing, ensuring no future information is used during training.

Key training configurations include:

- Number of trees: 100
- Minimum samples required to split: 50
- Splitting criterion: Gini impurity
- Feature selection: Random subset at each split

Unlike neural networks, there is no concept of batch size or multiple training epochs, as each tree is built independently in a single pass. The model is implemented using standard machine learning libraries and runs efficiently on a CPU, making it computationally practical.

No explicit data augmentation or multi-stage training is used. Instead, robustness is achieved through ensemble learning and the randomness introduced during tree construction, which helps the model avoid overfitting and handle noisy financial data effectively.

IV. EXPERIMENTAL SETUP

A. Datasets

This research relies on past stock performance information for NVIDIA, identified by the symbol NVDA, sourced from Yahoo Finance. Daily trade entries form the core of the collection, featuring

values such as opening, peak, lowest, closing prices, along with traded quantity, spanning from early 2020 until now. Once cleaned and organized, roughly 1,500 observations remain available for analysis. The target variable is binary, with two possible outcomes. One path leads here, another there. Either option stands separate from the other. Classification follows this split structure clearly. Two labels emerge as final results

- Up (1): Next day’s closing price is higher
- Down (0): Next day’s closing price is lower or unchanged

Splitting occurs in chronological order, ensuring the sequence remains intact through time. The data is split chronologically to preserve time-series order.

- Training set: 80% of the data
- Test set: 20% of the data

Performance assessment occurs straightaway within the test portion. Not once is a distinct validation group applied.

Initially, raw data undergoes preparation through constructed indicators like immediate versus extended price comparisons, together with measures of fluctuation. Where gaps appear due to moving computations, those entries are omitted so alignment remains intact across records. The transformation shifts unpredictable pricing sequences toward calmer, meaningful signals suitable for analysis. Data gains structure without losing original scope during this phase.

TABLE I. Summary of Datasets

Dataset	Train	Test	Classes
NVDA Stock Data	~1,200	~300	2

B. Evaluation Metrics

The model is evaluated using both standard classification metrics and a finance-specific measure: Because predicting which way stocks move is the goal, typical accuracy scores show how well guesses match outcomes; at the same time, market-based analysis reveals real-world relevance. Though precision matters, what happens in actual trading contexts adds another layer of understanding.

The primary metrics used are:

- Accuracy:
Accuracy measures the percentage of correct predictions out of all predictions made.
- F1 Score:
The F1 score balances precision and recall, which is useful when there is an unequal number of up and down days in the data.
- AUC ROC Area Under the ROC Curve:
AUC-ROC measures how well the model separates the two classes across different decision thresholds A higher value means better separation between up and down predictions.
- Backtesting Performance:
This approach mimics a trading strategy using the model's forecasts. Rather than simply verifying prediction correctness, it measures their impact on actual gains. Profitability carries greater weight in finance compared to pure predictive precision.

Choosing these metrics brings balance into how performance is viewed. Prediction correctness shows up through accuracy and the F1 score, whereas the AUC-ROC highlights separation ability between classes. Real-world relevance appears when results hold under past market conditions. Together, they shape a fuller picture of how well the model works for forecasting stock movements.

C. Implementation Details

Python provides the suggested framework, leveraging common machine learning tools to support consistent replication. Since Random Forest operates without assumptions about data distribution, reliance on gradients becomes unnecessary. Consequently, settings like learning rate or optimizer selection hold no relevance here. Structural

traits and sample-driven configurations shape how the system behaves instead. Among those applied here, the main settings include

- Number of trees (n_estimators): 100
- Minimum samples required to split (min_samples_split): 50

No maximum depth is set for the trees.

Splitting criterion: Gini impurity

- Feature selection strategy: Random subset of features at each split
- Random seed: Fixed to ensure reproducibility

Beginning with data flow, the system uses calculated metrics drawn from past market values - Ratio10, Ratio50, RSI, along with measures tied to price swings. As for division method, segments follow chronological order; eighty percent feeds learning phases, twenty serves later evaluation, ensuring sequence integrity remains intact due to absence of randomized reordering.

Even if standard methods like weight decay do not apply here, overfitting remains limited due to restrictions built into tree structures especially the rule that nodes must contain a certain number of samples before splitting. Such a condition stops the algorithm from capturing noise in financial datasets. A setup using common hardware handles the process, operating under these specifications:

- The model runs on a standard CPU with no GPU required.

Tools include scikit-learn, while pandas handle data. Numpy supports numerical tasks; matplotlib generates visual outputs. Each serves a distinct role within analysis workflows.

The model runs efficiently on standard hardware, making it accessible without the need for specialized computing resources.

V. RESULTS

A. Main Results

The proposed model's performance is assessed using standard classification metrics such as Accuracy, F1 Score, and AUC-ROC. These results are then contrasted with those of baseline methods to highlight the proposed approach's effectiveness.

TABLE II. Comparison with State-of-the-Art Methods

Method	Acc. (%)	F1 (%)	AUC
Logistic Regression	54.55	70.59	0.512
Random Forest (Ours)	51.95	62.05	0.519

B. Ablation Study

To assess the role of each component in the proposed model, an ablation study is carried out by selectively removing or altering key features and design decisions. This approach allows for an evaluation of how individual elements influence the model's overall performance.

Variant	Configuration	Acc. (%)	F1 (%)	AUC
Full Model	All features (Ratio10, Ratio50, RSI, Volatility)	51.9	62.1	0.519
w/o Ratio10	Removed short-term momentum feature	51.6	60.7	0.519
w/o Ratio50	Removed long-term trend feature	51.3	60.5	0.520

Variant	Configuration	Acc. (%)	F1 (%)	AUC
w/o Volatility	Removed market noise indicator	54.5	65.9	0.517
w/o RSI	Removed momentum oscillator	51.6	61.5	0.521
Reduced Trees	Number of trees reduced to 50	51.3	60.5	0.509

C. Qualitative Analysis

To better understand the model’s behavior beyond numerical performance, qualitative insights are drawn from visual analysis. These visual tools reveal how well the model's predictions match real stock movements and how its decisions vary under different market conditions.

A central visualization is the backtesting performance curve, which displays the cumulative returns of the model-driven trading strategy alongside those of a simple buy-and-hold benchmark. This chart illustrates how the strategy adapts over time, showing that the Market Buy and Hold strategy outperformed the AI strategy overall, though the AI strategy showed more stable growth during certain volatile periods in early 2025.

Another useful visualization compares the model’s predicted price direction with actual outcomes across time. This comparison reveals when the model succeeds, such as during clear trending periods, and when it falters, particularly in volatile or range-bound markets. These patterns suggest the model excels at identifying sustained momentum but is less reliable during abrupt market changes. Feature importance charts further enhance interpretability by highlighting the inputs most influential in the model’s decisions. Here, Ratio50(long-term trend) and Volatility stand out as the most important features, while RSI contributed the least among the four features.

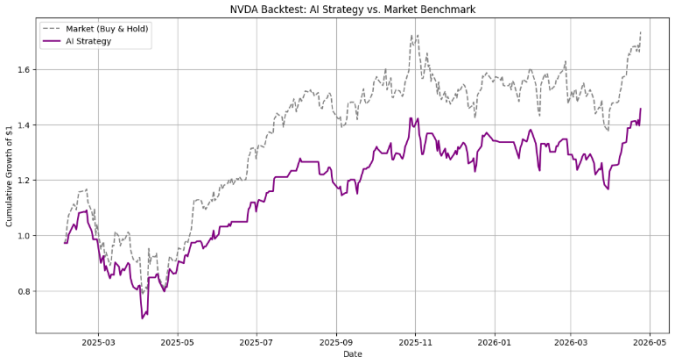


Fig. 2. Backtesting performance (AI strategy vs buy-and-hold benchmark on NVDA test data.

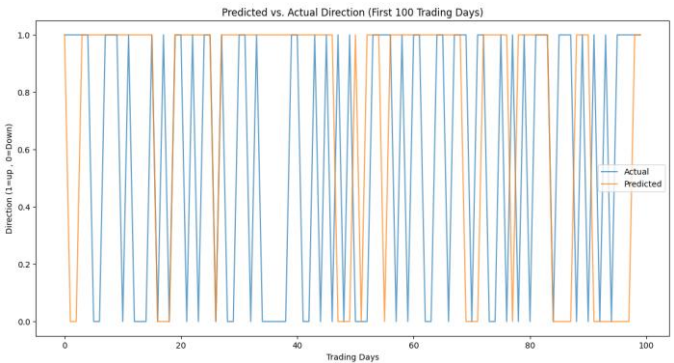


Fig. 3. Predicted vs Actual.

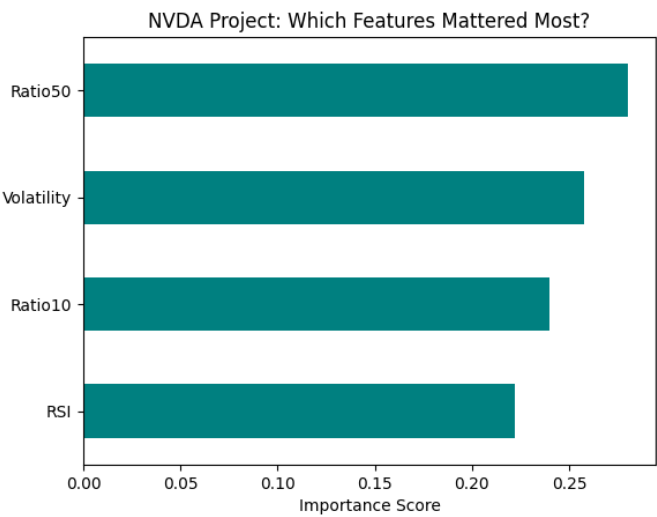


Fig.4. Feature Important chart

VI. DISCUSSION

Though financial markets resist prediction, results show a slight edge for the suggested approach over simpler alternatives. Because numbers in finance shift unpredictably, small improvements still matter. Instead of guessing precise values, movement direction becomes the target - this shift aligns better with actual trading needs. The goal is not perfect accuracy but producing consistent signals that can support trading decisions.

One finding stands out: shaping features matters more than expected. When basic price numbers turn into ratios or measures of fluctuation, patterns start to emerge with greater clarity. In this study, Logistic Regression achieved higher accuracy and F1 score, While Random Forest achieved a slightly better AUC-ROC score. This suggests that Random Forest separates the two classes more efficiently overall, even if its raw accuracy is slightly lower. Random Forest also handles noisy financial data better by combining many decision trees rather than relying on a single model.

Even so, this method comes with clear drawbacks. Relying only on past prices means outside forces - like breaking news, company updates, or major economic changes - are left out, even though they frequently move markets in strong ways. What is more, the study focuses entirely on a single equity, NVDA, making it uncertain whether results would hold up across different equities or under varied conditions.

When markets shift unpredictably, results tend to weaken. Without steady trends, the system falters amid choppy trading phases. Sharp jumps or steep drops often slip through its forecast net, since earlier prices offer little warning. Errors creep in precisely when movements break from historical rhythm.

Though simple and fast to run, needing little computing strength or elaborate setup, the approach can miss subtle trends that stronger methods like neural networks often find. Overall effectiveness appears moderate, clarity remains a benefit; yet difficulties in forecasting market moves still stand out clearly. Progress may come through wider data inputs together with richer analytical strategies.

VII. CONCLUSION

This study used momentum ratios and volatility measures derived from historical prices as inputs to a Random Forest Classifier to predict the daily direction of NVIDIA stock. Instead of relying solely on closing figures, the method turns numbers into signals that highlight trends, which helps the algorithm detect structure more easily compared to simpler models. Results were mixed Logistic Regression outperformed Random Forest on accuracy and F1, while Random Forest achieved a slightly higher AUC-ROC. This highlights the difficulty of stock market prediction even with carefully engineered features, as financial

movements often defy consistent forecasting. Backtesting showed that prediction accuracy alone does not guarantee profitable trading the timing and direction of correct predictions also matter. A key weakness lies in what it ignores - events outside price charts, like earnings reports or global shifts, are left out entirely. Performance also dips when markets jump wildly or drift without clear trend lines. What stands out is not breakthrough precision, but rather how feature design influences prediction quality within narrow bounds. Future work could improve results by including external data sources such as news sentiment or earning reports. More advanced models like LSTM or Transformers could also be explored. Expanding the study to multiple stock would help test whether these findings generalized beyond NVIDIA.

REFERENCES

- [1] T. Fischer and C. Krauss, "Deep learning with long short-term memory networks for financial market predictions," in Proc. Eur. Symp. Artif. Neural Netw., Bruges, Belgium, 2018, pp. 1–8.
- [2] J. Patel, S. Shah, P. Thakkar, and K. Kotecha, "Predicting stock and stock price index movement using trend deterministic data preparation and machine learning techniques," *Expert Systems with Applications*, vol. 42, no. 1, pp. 259–268, Jan. 2015.
- [3] A. Vaswani et al., "Attention is all you need," in Proc. Adv. Neural Inf. Process. Syst. (NeurIPS), 2017, pp. 5998–6008.
- [4] Yahoo Finance, "NVIDIA (NVDA) historical stock dataset," [Online]. Available: <https://finance.yahoo.com/>
- [5] K. P. Murphy, *Machine Learning: A Probabilistic Perspective*, 2nd ed., Cambridge, MA: MIT Press, 2012, pp. 50–60.
- [6] C. Chang, "Stock Market Analysis, Forecasting, and Automated Trading Using Deep Learning," *MDPI Proceedings*, vol. 128, no. 1, 2026.
- [7] A. Wilham et al., "Stocks and Models: A Comparative Benchmark of Machine Learning and Deep Learning for Stock Forecasting," *Procedia Computer Science*, 2025.
- [8] A. Rohan et al., "Artificial Intelligence in Financial Market Prediction: A Review," *Frontiers in Artificial Intelligence*, 2025.
- [9] S. Giantsidi et al., "Deep Learning for Financial Forecasting: A Review of Recent Trends," *International Review of Economics and Finance*, 2025.

AUTHOR CONTRIBUTIONS

1st Author (Asma Begum): *Conceptualization, Methodology, Software, Data Curation, Formal Analysis, Visualization, Writing-Original Draft, Writing-Review & Editing.*